INTERNSHIP PROJECT REPORT

Minor Projects in Very Large Scale Integration (VLSI)

Submitted by :

Name : SATYANARAYANA DHARSHINI
Department : BE - ECE
College : SATHYABAMA UNIVERSITY OF SCIENCE AND TECHNOLOGY
Internship Company : EDIGLOBE

LIST OF MINOR PROJECTS

1. Realize JK-FF using D-FF and write the Verilog code for a JK Flip-Flop, and its testbench. Use all possible combinations of inputs to test its working.
2. Write the hardware description of a 4-bit mod-13 counter and test it.
3. Design the Verilog HDL code for a 4-bit up down counter and test it using testbench.
4. Realize the function using 4x1 multiplexer module f(a,b,c)= (a+b′)(b′+c)$f(a,b,c)=(a+b')(b'+c)$.Find the minterms of the function realized by the circuit shown, and write the HDL for the function.
5. Simulate the UPDOWN counter counting 4 twice while down count using SystemVerilog enum datatype

PROJECT OVERVIEW

1. **JK Flip-Flop using D Flip-Flop :**

This project focuses on the design and verification of a JK flip-flop using a D flip-flop. The JK flip-flop is an important sequential circuit used in digital electronics because it can perform set, reset, hold, and toggle operations depending on the input conditions. In this project, the JK behavior is realized through the logic of a D flip-flop, showing how one type of flip-flop can be implemented using another. The design helps in understanding the relationship between different sequential circuits and improves knowledge of flip-flop conversions. The circuit is described using Verilog HDL and verified through simulation.

2. **4-bit Mod-13 Counter :**

This project deals with the design of a 4-bit mod-13 counter. A counter is a sequential circuit that changes its output in a specific order on every clock pulse. In a mod-13 counter, the sequence counts from 0 to 12 and then returns to 0. This type of counter is useful in timing, digital clocks, control systems, and sequence generation. The project demonstrates how a counter can be designed using Verilog HDL and how the counting sequence can be controlled by reset and clock signals. The working of the circuit is confirmed through simulation waveform.

3. **4-bit Up/Down Counter :**

This project presents the design of a 4-bit up/down counter using Verilog HDL. An up/down counter is a sequential circuit that can count both upward and downward depending on a control input. When the control signal is set for up counting, the output increases with each clock pulse, and when it is set for down counting, the output decreases with each clock pulse. This counter is useful in digital systems where forward and reverse counting are required. The project helps in understanding the working of sequential logic, control inputs, and binary counting. The final design is tested and verified using simulation.

4. **Function Realization Using 4x1 MUX :**

This project focuses on realizing a Boolean function using a 4x1 multiplexer. Multiplexers are important combinational circuits used for selecting one of many input signals and forwarding it to a single output line. In this project, the given Boolean expression is implemented using a 4x1 MUX by choosing suitable select lines and input combinations. This method simplifies logic circuit design and shows how a complex function can be implemented using a smaller hardware block. The project is implemented in Verilog HDL and verified through simulation results.

5. **UPDOWN Counter Counting 4 Twice During Down Count Using SystemVerilog Enum Datatype**

This project presents the design of an up/down counter using SystemVerilog enum datatype. The counter operates as an up/down counter based on a control signal, but it includes a special condition during down counting. When the count reaches 4 while counting down, the value 4 is displayed twice before the counter continues to decrement. This behavior is controlled using enum-based states, which makes the design easier to understand and manage. The project demonstrates the use of finite state machine concepts, sequential logic, and SystemVerilog enumeration. The output is checked through waveform simulation to confirm the required counting pattern.

PROJECT-1

1. **Realize JK-FF using D-FF and write the Verilog code for a JK Flip-Flop, and its testbench. Use all possible combinations of inputs to test its working.**

Aim :

To realize a JK flip-flop using a D flip-flop and verify its working for all possible input combinations.

Theory :

A JK flip-flop is a basic sequential circuit used in digital electronics for storing one bit of data. In this project, the JK flip-flop is realized using a D flip-flop by converting the JK input behavior into equivalent D input logic. The design helps in understanding how one flip-flop can be implemented using another type of flip-flop. A testbench is used to apply all possible input combinations and verify the output behavior. The waveform confirms the set, reset, hold, and toggle operations of the JK flip-flop. This project improves understanding of sequential logic design and simulation.

Expected Output :

- The output should follow the JK truth table correctly.
- When J = 0 and K = 0, the output should hold its previous state.
- When J = 0 and K = 1, the output should reset to 0.
- When J = 1 and K = 0, the output should set to 1.
- When J = 1 and K = 1, the output should toggle on every clock pulse.
- The waveform should clearly show hold, reset, set, and toggle operations.

VERILOG , TESTBENCH CODE :

testbench.sv

```
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("jkff.vcd");
        $dumpvars(0, tb_jk_using_dff);

        clk = 0; rst = 1; j = 0; k = 0;
        #10 rst = 0;

        j = 0; k = 0; #10;
        j = 0; k = 1; #10;
        j = 1; k = 0; #10;
        j = 1; k = 1; #10;
        j = 0; k = 0; #10;
        j = 1; k = 0; #10;
        j = 1; k = 1; #10;

        $finish;
    end
endmodule
```

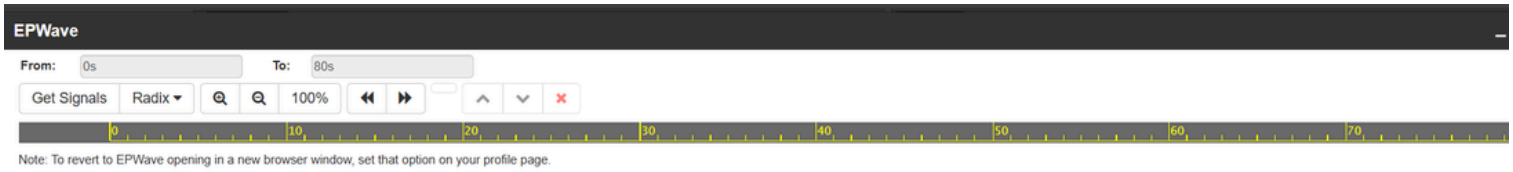
design.sv

```
module jk_using_dff(
    input clk,
    input rst,
    input j,
    input k,
    output reg q
);

wire d;
assign d = (j & ~q) | (~k & q);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else
        q <= d;
end

endmodule
```

Log | Share

```
[2026-04-14 06:18:12 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile jkff.vcd opened for output.
testbench.sv:30: $finish called at 80 (1s)
Finding VCD file...
./jkff.vcd
[2026-04-14 06:18:13 UTC] Opening EPWave...
Done
```

WAVEFORM :



OBSERVATION :

- The output follows the JK truth table correctly.
- The flip-flop holds its previous state when J = 0 and K = 0.
- It resets when J = 0 and K = 1.
- It sets when J = 1 and K = 0.
- It toggles when J = 1 and K = 1.

RESULT :

- The JK flip-flop was successfully realized using a D flip-flop.
- The simulation verified the correct operation for all input combinations.

PROJECT-2

2. **Write the hardware description of a 4-bit mod-13 counter and test it.**

Aim :

To design a 4-bit mod-13 counter and test its working using a testbench

Theory :

A counter is a sequential digital circuit that changes its output state with every clock pulse. It is one of the most important building blocks in digital systems because it is used for counting events, generating timing sequences, and controlling repetitive operations. A mod-13 counter is a special type of counter that counts from 0 to 12 and then returns back to 0. Since it has 13 valid states, it is called a modulo-13 counter. In this project, a 4-bit counter is used because 4 bits are enough to represent values from 0 to 15, which covers the required 13 states.
The counter is designed using Verilog HDL and works synchronously with the clock signal. On each positive edge of the clock, the output increases by one. When the count reaches 12, the next clock pulse resets the output back to 0. This makes the circuit useful in applications where counting needs to stop after a fixed number of states and then repeat again. A reset input is also included so that the counter can be initialized to zero whenever required.

Expeced Output :

- The counter should start from 0000.
- It should count upward on every clock pulse.
- It should count from 0000 to 1100.
- After reaching 1100, the next clock pulse should reset it to 0000.
- The waveform should show correct mod-13 counting behavior.

VERILOG , TESTBENCH CODE :

testbench.sv

```verilog
    wire [3:0] q;

    mod13_counter dut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("mod13.vcd");
        $dumpvars(0, tb_mod13_counter);

        clk = 0; rst = 1;
        #10 rst = 0;

        repeat(20) @(posedge clk);

        $finish;
    end
endmodule
```

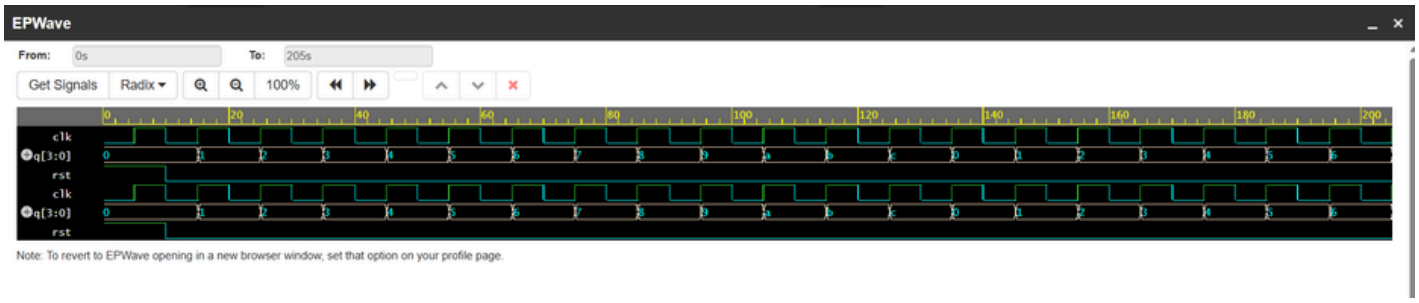
design.sv

```verilog
module mod13_counter (
    input clk,
    input rst,
    output reg [3:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 4'b0000;
    else if (q == 4'b1100)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end

endmodule
```

Log | Share

```
[2026-04-14 06:20:38 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile mod13.vcd opened for output.
testbench.sv:22: $finish called at 205 (1s)
Finding VCD file...
./mod13.vcd
[2026-04-14 06:20:39 UTC] Opening EPWave...
Done
```

WAVWFORM :



OBSERVATION :

- The counter starts from 0000 and increments on every clock pulse.
- It counts correctly up to 1100, which is decimal 12.
- After reaching 1100, it resets back to 0000.
- The waveform shows proper mod-13 counting behavior.

RESULT :

- The 4-bit mod-13 counter was successfully designed and tested.
- The simulation output matched the expected counting sequence.

PROJECT-3

3. **Design the Verilog HDL code for a 4-bit up down counter and test it using testbench.**

Aim :

To design a 4-bit up/down counter and test it using a testbench.

Theory :

An up/down counter is a sequential digital circuit that can count in both increasing and decreasing order depending on the control input. It is widely used in digital systems where values need to be incremented or decremented under clock control, such as timers, event counters, and simple control units. In this project, a 4-bit up/down counter is designed using Verilog HDL, which means the output can represent values from 0 to 15 in binary form. The counter changes its state on every clock pulse, so the output updates synchronously with the clock. When the control input is set for up counting, the counter increases by one on each active clock edge. When the control input is changed to down counting, the counter decreases by one on each active clock edge. A reset input is usually included so that the counter can be initialized to zero before operation begins. This makes the design more practical and easy to test during simulation. The 4-bit width is enough to show a complete binary sequence and to demonstrate both incrementing and decrementing behavior clearly.

Expeced Output :

- The counter should count upward when the up/down control is set to up mode.
- The counter should count downward when the control is set to down mode.
- The output should change on every clock pulse.
- The waveform should show both increment and decrement operations.
- The 4-bit output should remain valid throughout the simulation.

VERILOG , TESTBENCH CODE :

testbench.sv

```verilog
        .clk(clk),
        .rst(rst),
        .ud(ud),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("updown.vcd");
        $dumpvars(0, tb_updown_counter);

        clk = 0; rst = 1; ud = 1;
        #10 rst = 0;

        repeat(8) @(posedge clk);
        ud = 0;
        repeat(8) @(posedge clk);

        $finish;
    end
endmodule
```

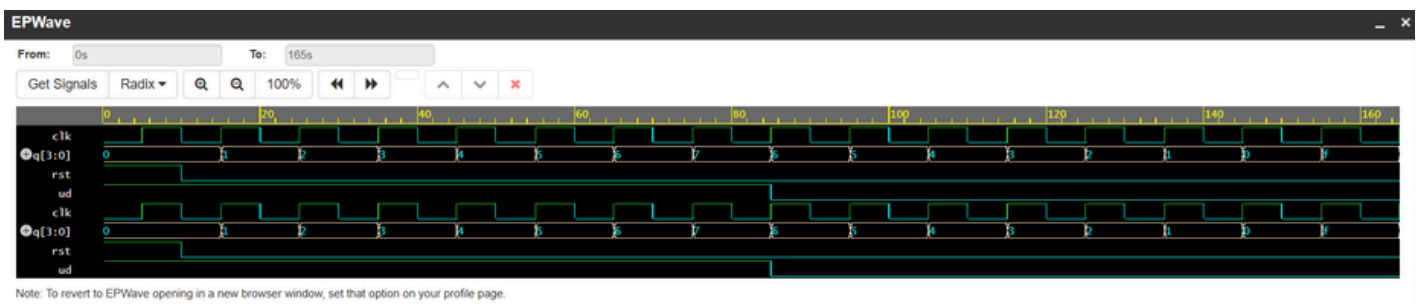
design.sv

```verilog
module updown_counter (
    input clk,
    input rst,
    input ud,
    output reg [3:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 4'b0000;
    else if (ud)
        q <= q + 1'b1;
    else
        q <= q - 1'b1;
end

endmodule
```

Log | Share

```
[2026-04-14 06:22:23 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile updown.vcd opened for output.
testbench.sv:25: $finish called at 165 (1s)
Finding VCD file...
./updown.vcd
[2026-04-14 06:22:25 UTC] Opening EPWave...
Done
```

WAVWFORM :



OBSERVATION :

- The counter increments when the control signal is set to up mode.
- The counter decrements when the control signal is set to down mode.
- The output changes correctly on every clock pulse.
- The waveform clearly shows both up and down counting operation.

RESULT :

- The 4-bit up/down counter was successfully implemented.
- The simulation confirmed correct counting in both directions.

PROJECT-4

4. **Realize the function using 4x1 multiplexer module f(a,b,c)=(a+b′)(b′+c)f(a,b,c)=(a+b′)(b′+c).Find the minterms of the function realized by the circuit shown, and write the HDL for the function.**

Aim :

To design a 4-bit up/down counter and test it using a testbench.

Theory :

A multiplexer is a combinational circuit that selects one of several input lines and forwards it to the output based on the select inputs. It is one of the most useful building blocks in digital design because many logic functions can be implemented efficiently using a multiplexer. In this project, the given Boolean function f(a,b,c)=(a+b′)(b′+c)f(a,b,c)=(a+b′)(b′+c) is realized using a 4x1 multiplexer. This means the logic expression is converted into a form that can be directly implemented using the MUX structure.
First, the Boolean function is simplified so that it becomes easier to map into the multiplexer inputs. Then, two variables are chosen as select lines, and the remaining variable is used on the data input side wherever required. This method reduces the complexity of direct gate-level implementation and shows how a multiplexer can be used as a general logic realization device. It also helps in understanding the relation between Boolean expressions and hardware design.

Expeced Output :

- The output f should match the given Boolean function.
- The multiplexer should select the correct input based on the select lines.
- For every input combination, the output should be correct.
- The waveform should verify the expression f(a,b,c)=(a+b′)(b′+c)f(a,b,c)=(a+b′)(b′+c).
- The design should show proper MUX-based logic realization.

VERILOG , TESTBENCH CODE :

testbench.sv

```
        .a(a),
        .b(b),
        .c(c),
        .f(f)
    );

    initial begin
        $dumpfile("muxfunc.vcd");
        $dumpvars(0, tb_mux4x1_function);

        a=0; b=0; c=0; #10;
        a=0; b=0; c=1; #10;
        a=0; b=1; c=0; #10;
        a=0; b=1; c=1; #10;
        a=1; b=0; c=0; #10;
        a=1; b=0; c=1; #10;
        a=1; b=1; c=0; #10;
        a=1; b=1; c=1; #10;

        $finish;
    end
endmodule
```

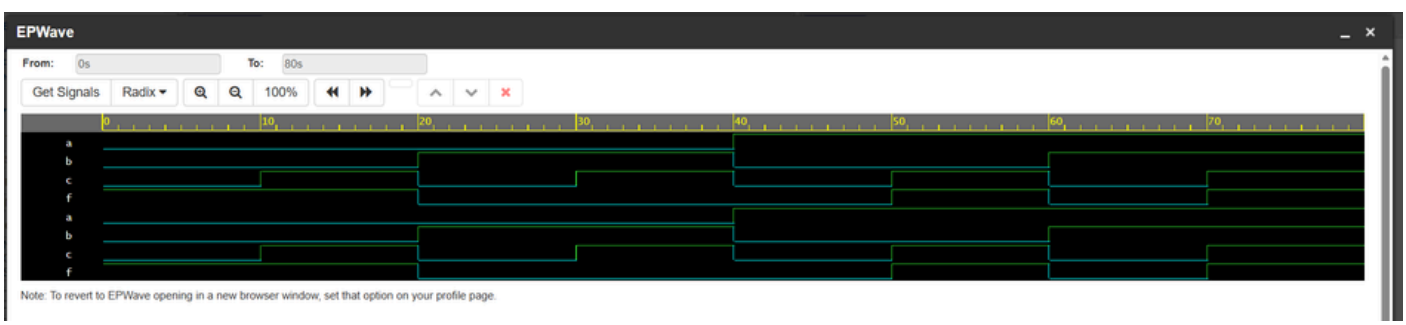
design.sv

```
module mux4x1_function(
    input a,
    input b,
    input c,
    output reg f
);

always @(*) begin
    case ({a,b})
        2'b00: f = 1'b1;
        2'b01: f = 1'b0;
        2'b10: f = c;
        2'b11: f = c;
    endcase
end

endmodule
```

Log | Share

```
[2026-04-14 06:23:46 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile muxfunc.vcd opened for output.
testbench.sv:25: $finish called at 80 (1s)
Finding VCD file...
./muxfunc.vcd
[2026-04-14 06:23:47 UTC] Opening EPWave...
Done
```

WAVWFORM :



OBSERVATION :

- The multiplexer output matches the given Boolean function.
- The correct data input is selected based on the select lines.
- The waveform shows proper output for all input combinations.
- The logic realization using 4x1 MUX works correctly.

RESULT :

- The Boolean function was successfully realized using a 4x1 multiplexer.
- The simulation verified the correct output for all test cases.

PROJECT-5

5. **Simulate the UPDOWN counter counting 4 twice while down count using SystemVerilog enum datatype**

Aim :

To design an UPDOWN counter that counts 4 twice while down counting using SystemVerilog enum datatype.

Theory :

A counter is a sequential digital circuit that changes its output according to the clock signal. Unlike combinational circuits, a counter remembers its previous state, so it is built using memory elements such as flip-flops. Counters are widely used in digital systems for timing, sequencing, event counting, and control applications.
An up/down counter is a special type of counter that can either increment or decrement based on a control input. When the control signal indicates up counting, the counter increases its value by 1 on every clock edge. When it indicates down counting, the counter decreases by 1 on every clock edge. This makes the same circuit useful for both forward and backward counting operations. In this project, the counter is designed in SystemVerilog using the enum datatype. An enum is used to define named states in a simple and readable way. Instead of writing raw numeric state values, the designer can use names such as NORMAL, HOLD1, and HOLD2. This makes the code easier to understand, debug, and maintain.

Expeced Output :

- The counter should count normally in up mode.
- The counter should count normally in down mode.
- While down counting, the value 4 should appear twice.
- The waveform should clearly show the repeated 4 during down count.
- The output should confirm correct enum-based control behavior.

VERILOG , TESTBENCH CODE :

testbench.sv

```
        .rst(rst),
        .ud(ud),
        .q(q)
    );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("enum_updown.vcd");
        $dumpvars(0, tb_updown_enum_counter);

        clk = 0; rst = 1; ud = 0;
        #10 rst = 0;

        repeat(12) @(posedge clk);

        ud = 1;
        repeat(6) @(posedge clk);

        $finish;
    end
endmodule
```

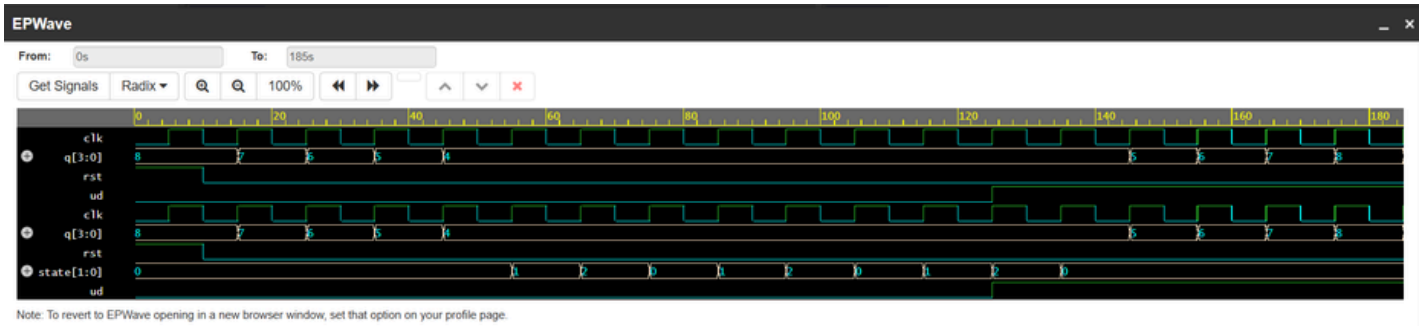
design.sv

```
            NORMAL: begin
                if (ud)
                    q <= q + 1;
                else if (q == 4) begin
                    q <= 4;
                    state <= HOLD1;
                end else
                    q <= q - 1;
            end
            HOLD1: begin
                q <= 4;
                state <= HOLD2;
            end
            HOLD2: begin
                q <= 4;
                state <= NORMAL;
            end
        endcase
    end
end

endmodule
```

Log | Share

```
[2026-04-14 06:25:13 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile enum_updown.vcd opened for output.
testbench.sv:26: $finish called at 185 (1s)
Finding VCD file...
./enum_updown.vcd
[2026-04-14 06:25:14 UTC] Opening EPWave...
Done
```

WAVWFORM :



OBSERVATION :

- The counter works normally in both up mode and down mode.
- During down counting, the value 4 appears twice as required.
- The waveform shows the repeated count behavior clearly.
- The enum-based control improves the structure of the design.

RESULT :

- The UPDOWN counter using SystemVerilog enum datatype was successfully designed.
- The simulation verified the special counting condition correctly.